

EduFee

Student Fee Collection & Reconciliation System

1. Project Overview

EduFee is a web-based Student Fee Collection and Reconciliation System designed for educational institutions to manage student fees, concessions, installments, payments, outstanding balances, and payment reconciliation from a centralized platform.

The system provides separate functionality for students and authorized institutional staff. Students can view their fee obligations, invoices, installments, payment history, and outstanding balance. Accounts staff can manage fee structures, concessions, invoices, payments, reconciliation, and financial records.

The primary focus of the system is maintaining **financial consistency, traceability, and reliable payment state management** rather than simply recording successful payments.

The system also handles real-world payment scenarios such as:

- Successful payments
 - Failed payments
 - Pending payments
 - Reversed payments
 - Partial payments
 - Installment-based payments
 - Concessions and discounts
 - Outstanding balances
 - Duplicate payment callbacks
 - Payment amount mismatches
 - Reconciliation differences
 - Audit history of financial changes
-

2. Problem Statement

Educational institutions may have multiple fee heads and different fee structures for students based on courses, semesters, academic years, concessions, and installment plans.

A fee management system therefore needs to answer questions such as:

- How much does a student need to pay?
- Which fee heads contribute to the amount?
- Has a concession been applied?

- What installments are due?
- How much has actually been paid?
- Was a payment successful, failed, pending, or reversed?
- What amount is still outstanding?
- Does the internal payment record match the payment gateway record?
- Who changed an important financial record?
- Can a previous transaction be traced and verified?

EduFee addresses these problems through a structured workflow connecting **fee structures** → **invoices** → **installments** → **payments** → **reconciliation** → **audit records**.

The design gives particular importance to payment-state handling and reconciliation because a payment attempt should not automatically be treated as a successful financial transaction.

3. Project Objectives

The main objectives of EduFee are:

1. **Centralize fee management**
Maintain fee heads, fee structures, and fee components in one system.
2. **Manage student invoices**
Generate invoices containing the applicable fee amount, concessions, due date, and payment status.
3. **Support installments**
Allow an invoice to be divided into multiple installments and track each installment independently.
4. **Handle different payment states**
Maintain explicit states such as **INITIATED**, **PENDING**, **SUCCESS**, **FAILED**, and **REVERSED**.
5. **Calculate outstanding amounts accurately**
Only effective successful payments should reduce the student's outstanding balance.
6. **Support concessions**
Allow approved fixed or percentage-based concessions to be applied to applicable fees.
7. **Provide payment history**
Preserve successful, failed, pending, and reversed payment attempts for traceability.
8. **Provide reconciliation**
Compare internal payment records with gateway-side transaction information and identify mismatches.
9. **Maintain auditability**
Record important financial modifications and the user responsible for them.
10. **Provide reporting and visibility**
Give authorized staff an overview of collections, outstanding amounts, payment statuses, and reconciliation results.

11. **Handle edge cases safely**

Prevent invalid amounts, duplicate transaction processing, incorrect state transitions, and inconsistent financial records.

4. Functional Requirements

EduFee is designed around the complete fee-management lifecycle, from defining fee structures to collecting and reconciling payments.

4.1 Student Management

The system maintains student information required for fee management.

Each student can have:

- Student ID
- Name
- Email
- Phone number
- Course
- Department
- Semester
- Academic year
- Account status

Students can log in and view their own financial information.

4.2 Fee Head Management

A **Fee Head** represents a particular category of fee charged by the institution.

Examples:

- Tuition Fee
- Examination Fee
- Library Fee
- Laboratory Fee
- Hostel Fee
- Transportation Fee

Accounts staff can create, update, activate, or deactivate fee heads.

A fee head can then be associated with one or more fee structures.

4.3 Fee Structure Management

A **Fee Structure** defines the fees applicable to a particular academic context.

For example:

B.Tech CSE — Semester 5 — Academic Year 2026–27

A fee structure consists of multiple fee components.

Example:

Fee Head	Amount
Tuition Fee	₹50,000
Examination Fee	₹2,000
Library Fee	₹1,000
Laboratory Fee	₹3,000
Gross Amount	₹56,000

The system calculates the total from the individual fee components instead of relying on a manually entered total.

4.4 Concession Management

The system supports student-specific concessions.

A concession can be:

- Fixed amount
- Percentage-based

Each concession contains a reason and approval information.

Only an approved concession affects the final invoice amount.

Calculation

For a fixed concession:

$$\text{Net Amount} = \text{Gross Amount} - \text{Concession Amount}$$

For a percentage concession:

$$\text{Concession Amount} = \text{Gross Amount} \times \text{Percentage} / 100$$

$$\text{Net Amount} = \text{Gross Amount} - \text{Concession Amount}$$

The system prevents the final payable amount from becoming negative.

4.5 Invoice Management

An invoice represents the student's financial obligation for a particular fee structure.

An invoice contains:

- Invoice number
- Student
- Fee structure
- Gross amount
- Concession amount
- Net amount
- Due date
- Payment status
- Creation timestamp

Possible invoice statuses include:

- DRAFT

- ISSUED
 - PARTIALLY_PAID
 - PAID
 - OVERDUE
 - CANCELLED
-

4.6 Installment Management

An invoice can be divided into multiple installments.

For example:

Invoice Amount: ₹60,000

Installment	Amount	Due Date
1	₹20,000	10-Jul-2026
2	₹20,000	10-Aug-2026
3	₹20,000	10-Sep-2026

The total of all installments must equal the invoice's net amount.

This prevents situations where the installment schedule does not represent the actual invoice amount.

4.7 Payment Management

The system records every payment attempt.

Supported payment methods include:

- UPI
- Card
- Net Banking
- Cash
- Bank Transfer

Payment statuses are explicitly maintained:

- INITIATED
- PENDING
- SUCCESS
- FAILED
- REVERSED

Only a **SUCCESS** payment contributes to the student's effective paid amount.

A failed or pending payment remains visible in payment history but does not reduce the outstanding balance.

4.8 Outstanding Amount

The student's outstanding amount is calculated using the financial records rather than being manually maintained.

Formula

Outstanding = Net Invoice Amount - Effective Successful Payments

For example:

- Invoice = ₹50,000
- Successful payments = ₹30,000

Therefore:

Outstanding = ₹50,000 - ₹30,000 = ₹20,000

A failed, pending, or reversed payment does not reduce the outstanding amount.

4.9 Payment History

Students and authorized staff can view payment history.

The history contains:

- Payment reference
- Invoice
- Amount
- Payment method
- Payment status
- Gateway transaction ID
- Initiation time
- Completion time

Failed transactions are retained instead of being deleted because they provide useful information for troubleshooting and auditability.

4.10 Reconciliation

Reconciliation compares the system's payment record against the corresponding gateway transaction.

The system records:

- Gateway reference
- System amount
- Gateway amount
- System status
- Gateway status
- Difference
- Reconciliation status
- Resolution information

Possible reconciliation statuses:

- MATCHED
- MISMATCH
- RESOLVED

A mismatch is flagged for review rather than silently modifying the internal transaction.

5. User Roles

EduFee uses role-based access to separate student functionality from administrative financial operations.

5.1 Student

Students can:

- Log in
- View their profile
- View invoices
- View installments
- View outstanding amount
- View payment history
- Make or simulate payments
- View payment status
- View receipts

Students should not be able to modify institutional fee structures or reconciliation records.

5.2 Accounts Staff

Accounts staff can:

- Manage students
- Manage fee heads
- Manage fee structures
- Create invoices
- Configure installments
- Review concessions
- View payments
- Perform reconciliation
- View reports
- Review financial records

5.3 Administrator

The administrator has broader system-level access.

Administrative functionality includes:

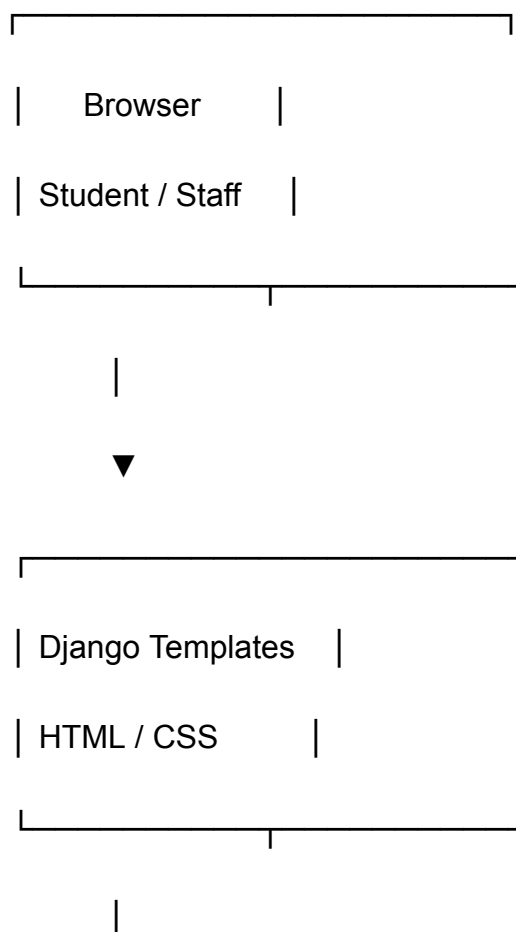
- User and role management
- Access to financial configuration
- Audit log visibility
- System monitoring
- Management reports

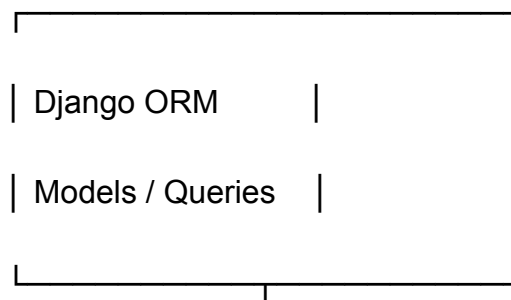
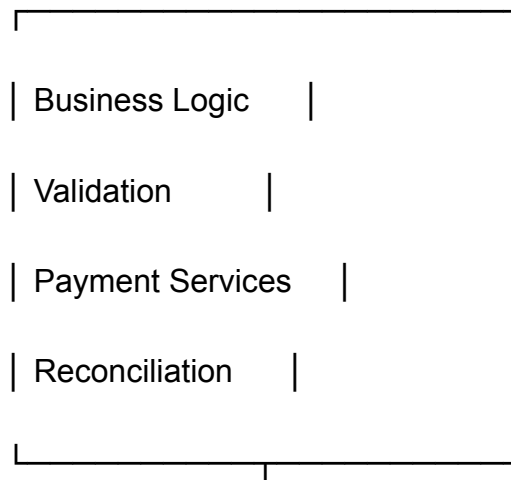
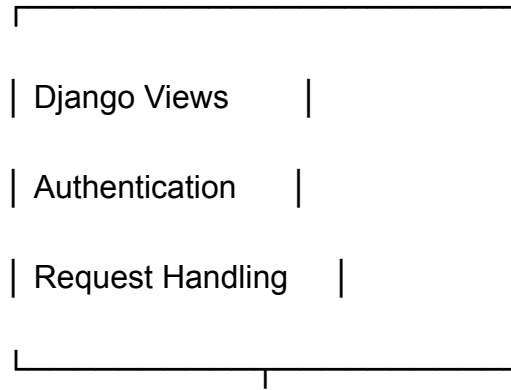
Role checks are enforced on the server side rather than relying only on hiding buttons in the interface.

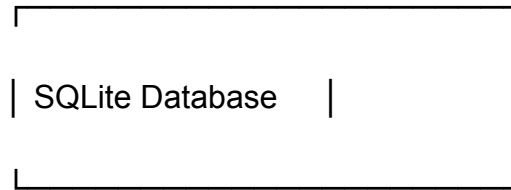
6. System Architecture

6.1 Architecture Overview

EduFee follows a layered Django web architecture.







6.2 Application Components

The project is divided into logical Django applications.

accounts

Responsible for:

- Authentication
- User roles
- Permissions
- Login/logout

students

Responsible for:

- Student profiles
- Academic information
- Student status

fees

Responsible for:

- Fee heads
- Fee structures
- Fee components
- Concessions
- Invoices
- Installments

payments

Responsible for:

- Payment records
- Payment states
- Payment history
- Payment validation

reconciliation

Responsible for:

- Gateway transaction comparison
- Mismatch detection
- Reconciliation status
- Resolution records

audit

Responsible for:

- Audit logs
- Financial change tracking
- User/action history

reports

Responsible for:

- Collection reports
- Outstanding reports
- Payment status reports
- Reconciliation reports

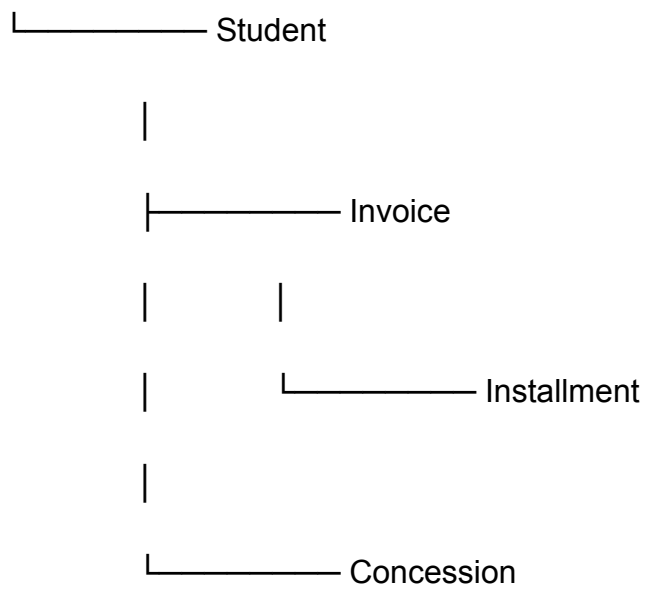
7. Database Design

The database follows a relational model where financial entities are connected through foreign-key relationships.

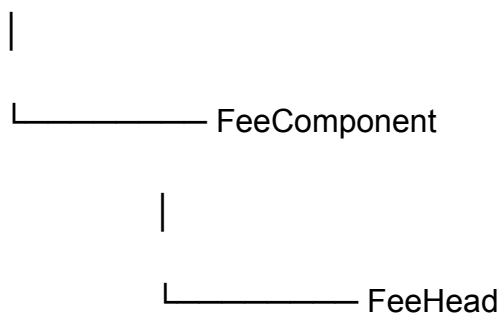
7.1 Main Entities

User

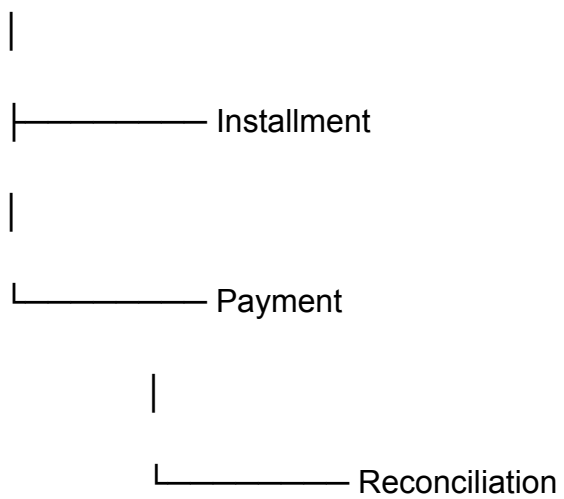
|



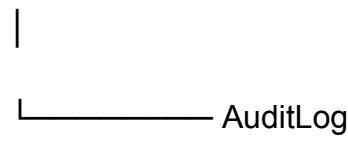
FeeStructure



Invoice



User



7.2 Student

Field	Purpose
student_id	Unique institutional identifier
user	Link to authentication account
name	Student name
email	Contact email
phone	Contact number
course	Academic course
department	Department
semester	Current semester

academic_year	Academic year
status	Active/inactive status
created_at	Creation timestamp

7.3 FeeHead

Field	Purpose
name	Fee category
description	Description
is_active	Whether fee head can be used
created_at	Creation timestamp

7.4 FeeStructure

Field	Purpose
-------	---------

<code>name</code>	Structure name
<code>course</code>	Applicable course
<code>semester</code>	Applicable semester
<code>academic_year</code>	Academic year
<code>is_active</code>	Active/inactive state
<code>created_at</code>	Creation timestamp

7.5 FeeComponent

A fee component connects a fee structure with a fee head.

Field	Purpose
<code>fee_structure</code>	Parent fee structure
<code>fee_head</code>	Fee category
<code>amount</code>	Amount charged

Example:

B.Tech CSE Semester 5

—	Tuition Fee	₹50,000
—	Exam Fee	₹2,000
—	Library Fee	₹1,000
└—	Lab Fee	₹3,000

7.6 Concession

Field	Purpose
student	Student receiving concession
fee_component	Applicable fee
concession_type	Fixed/percentage
value	Concession value
reason	Reason for concession
status	Approval state

approved_by Approving staff

approved_at Approval timestamp

7.7 Invoice

Field	Purpose
invoice_number	Unique invoice identifier
student	Invoice owner
fee_structure	Applicable fee structure
gross_amount	Total before concession
discount_amount	Applied concession
net_amount	Final payable amount
due_date	Payment due date

<code>status</code>	Invoice state
<code>created_at</code>	Creation timestamp
<code>updated_at</code>	Last modification

7.8 Installment

Field	Purpose
<code>invoice</code>	Parent invoice
<code>installment_number</code>	Sequence number
<code>amount</code>	Installment amount
<code>due_date</code>	Due date
<code>status</code>	Payment status
<code>created_at</code>	Creation timestamp

7.9 Payment

Field	Purpose
payment_reference	Internal unique reference
invoice	Related invoice
installment	Related installment
amount	Payment amount
payment_method	Payment method
status	Payment state
gateway_transaction_id	Gateway transaction identifier
initiated_at	Initiation time
completed_at	Completion time
created_by	User who initiated/recorded payment

7.10 Reconciliation

Field	Purpose
payment	Internal payment
gateway_reference	Gateway reference
system_amount	Amount recorded internally
gateway_amount	Amount received from gateway
system_status	Internal payment status
gateway_status	Gateway status
reconciliation_status	Matched/mismatch/resolved
difference	Amount difference
checked_at	Reconciliation time
resolved_at	Resolution time
resolved_by	Staff resolving mismatch

`resolution_note` Explanation of resolution

7.11 AuditLog

The audit log records important system actions.

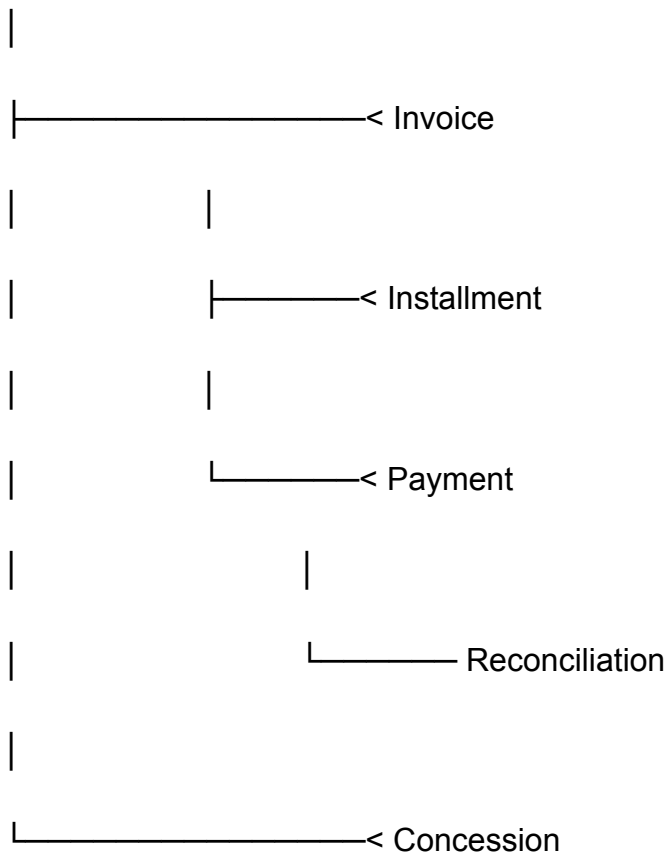
Field	Purpose
<code>user</code>	User performing action
<code>action</code>	Action performed
<code>entity_type</code>	Affected entity
<code>entity_id</code>	Affected record
<code>old_value</code>	Previous state
<code>new_value</code>	New state
<code>description</code>	Human-readable explanation

timestamp Time of action
p

8. Core Data Relationships

The main relationships can be summarized as follows:

Student



FeeStructure





Relationship Explanation

Student → Invoice

One student can have multiple invoices.

Invoice → Installment

One invoice can contain multiple installments.

Invoice → Payment

An invoice can have multiple payment transactions.

Payment → Reconciliation

A payment can have a reconciliation record representing its comparison with the gateway transaction.

FeeStructure → FeeComponent

A fee structure consists of multiple fee components.

FeeComponent → FeeHead

Each component represents a particular fee category.

Student → Concession

A student can receive multiple concessions subject to the applicable business rules.

This relational design keeps financial entities separated and makes it possible to trace a student's financial journey from the original fee structure through the final payment and reconciliation record.

AI Usage Report

1. AI Tool Used

Tool: ChatGPT

Purpose: Product planning, system design, implementation assistance, debugging, validation, documentation, and review.

AI assistance was used throughout the development of the EduFee Student Fee Collection & Reconciliation System. The generated suggestions and code were reviewed, adapted, and tested before being incorporated into the project.

2. How AI Was Used

AI assistance was used for the following activities:

Product Understanding

ChatGPT was used to break down the assignment requirements into functional modules and identify the important business workflows.

The assignment was analyzed to identify key requirements such as:

- Fee heads
- Fee structures
- Discounts and concessions
- Invoices
- Installments
- Payment states
- Outstanding amounts
- Payment history
- Reconciliation
- Auditability
- User roles
- Reporting
- Edge-case handling

System Design

ChatGPT was used to design the overall application architecture and database structure.

The proposed architecture was based on:

Browser



Django Templates



Django Views



Business Logic



Django ORM



Database

The AI was also used to identify the major entities and relationships required for the system.

Database Design

AI assistance was used while designing models such as:

- Student
- FeeHead
- FeeStructure
- FeeComponent
- Concession
- Invoice
- Installment
- Payment
- Reconciliation
- AuditLog

The generated model suggestions were reviewed and adapted according to the actual implementation requirements.

3. Example Useful Prompt

One of the prompts used during development was:

"Design a Django-based student fee collection and reconciliation system. It should support multiple fee heads, fee structures, concessions, invoices, installments, payments with INITIATED/PENDING/SUCCESS/FAILED/REVERSED states, outstanding amount calculation, payment history, reconciliation, audit logs, user roles, and reporting. Focus on financial consistency, validation, and edge cases."

This prompt helped establish the initial architecture and identify the important business entities and workflows.

4. Code Generation

AI was used to assist with implementation of Django components, including:

- Django models
- Views
- URL configurations
- HTML templates
- Forms
- Validation logic
- Dashboard components
- Database relationships
- Business logic
- Utility functions

The generated code was not treated as automatically correct. It was reviewed and modified according to the actual project structure.

5. Code Modification

AI-generated suggestions were adapted to the existing EduFee project.

Modifications included:

- Adjusting model relationships
- Changing field names to match the project
- Adapting views to existing URLs and templates
- Updating template structure
- Adding project-specific validation
- Modifying business logic
- Fixing database-related issues
- Adjusting UI elements to match the application

The final implementation was tested within the actual Django project rather than using generated code without verification.

6. Debugging and Problem Solving

ChatGPT was also used as a debugging assistant.

When errors occurred, relevant error messages and code were provided to identify possible causes.

The debugging process generally followed:

Error



Analyze traceback



Identify probable cause



Propose fix



Apply fix



Run application



Verify behavior

This approach helped identify configuration, database, model, URL, template, and validation issues during development.

7. Example of Incorrect AI Output

AI-generated code was not always directly suitable for the existing project.

For example, generated code sometimes assumed:

- A model field that did not exist in the current project
- A different URL name
- A different application structure
- A different relationship between models
- A different template path

Such suggestions were identified when the application produced errors or when the generated implementation did not match the existing project structure.

Instead of directly accepting the generated solution, the code was compared with the actual project files and modified accordingly.

8. How Incorrect Output Was Identified

Incorrect or unsuitable AI suggestions were identified through:

1. Django traceback/error messages
2. Application runtime behavior
3. Database migration errors
4. Template rendering errors
5. URL resolution errors
6. Manual testing of workflows
7. Checking whether business rules were correctly enforced

8. Comparing generated code with the existing project structure

This validation was important because generated code could be syntactically valid but still incorrect for the project's actual requirements.

9. Business Logic Validation

AI was particularly useful for identifying edge cases in financial workflows.

Examples considered include:

Failed Payment

A failed payment should remain in transaction history but should not reduce the outstanding amount.

Pending Payment

A pending payment should not immediately be treated as successful.

Reversed Payment

A previously successful payment that is reversed should no longer reduce the effective paid amount.

Duplicate Transaction

Repeated gateway callbacks should not create duplicate successful transactions.

Overpayment

The system should validate payment amounts and prevent an invalid payment from exceeding the outstanding amount where applicable.

Reconciliation Mismatch

If the amount recorded by the system differs from the gateway amount, the transaction should be flagged for reconciliation rather than silently changing the financial record.

10. Human Validation

AI was used as an engineering assistant rather than as an autonomous development system.

The developer remained responsible for:

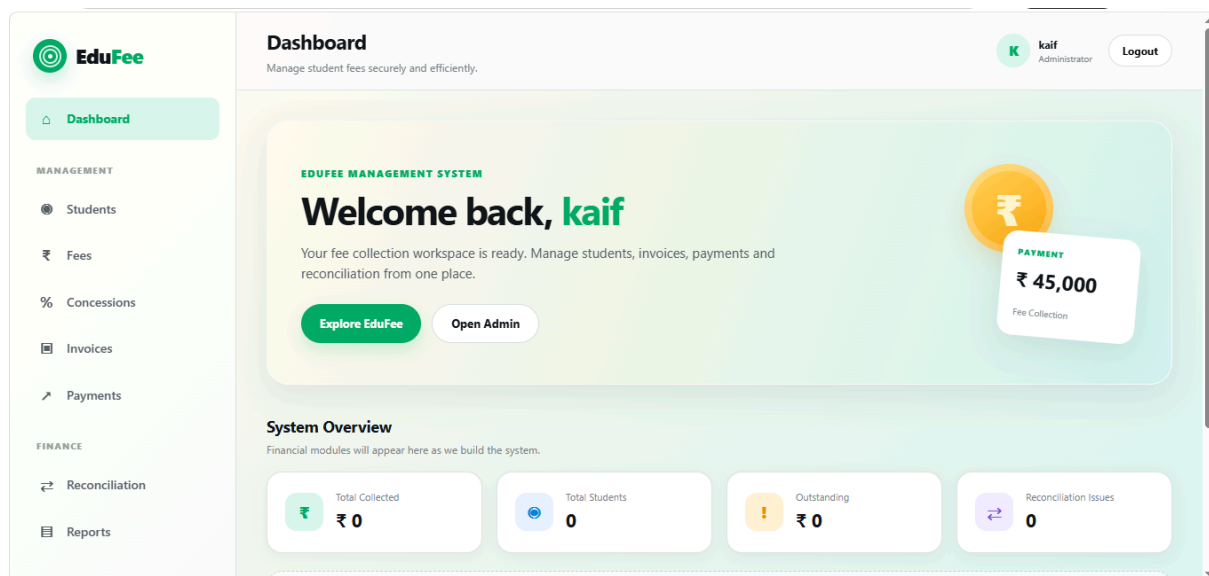
- Selecting the final architecture
- Reviewing generated code
- Integrating code into the project
- Testing functionality
- Validating financial calculations
- Identifying incorrect suggestions
- Making final engineering decisions
- Verifying that the implementation satisfied the assignment requirements

11. AI Usage Summary

AI assistance was primarily used to accelerate development and improve the reasoning process around architecture, implementation, debugging, and edge cases.

The final application was assembled and validated within the actual Django project. AI-generated suggestions were treated as proposals that required human review and testing before adoption.

Project Screenshot





Dashboard

MANAGEMENT

Students

Fees

Concessions

Invoices

Payments

FINANCE

Reconciliation

Reports

Fee Heads

Manage student fees securely and efficiently.

K

kaif

Administrator

Logout

FEE CONFIGURATION

Fee Heads

Manage reusable categories of student fees.

+ Add Fee Head

Fee Head Configuration

4 fee heads

NAME	DESCRIPTION	STATUS	CREATED
Examination Fee	University examination charges	Active	24 Sep 2026
Laboratory Fee	Laboratory usage and maintenance charges	Active	24 Sep 2026
Library Fee	Library and digital resource charges	Active	24 Sep 2026
Tuition Fee	Semester tuition charges	Active	24 Sep 2026



Dashboard

MANAGEMENT

Students

Fees

Concessions

Invoices

Payments

FINANCE

Reconciliation

Reports

Dashboard

Manage student fees securely and efficiently.

K

kaif

Administrator

Logout

Invoices

Manage student fee invoices.

+ Create Invoice

INVOICE	STUDENT	FEE STRUCTURE	GROSS	DISCOUNT	NET	DUE DATE	STATUS
INV-2026-000002	2210013135065 kaif	B.Tech CSE - Semester 5 - 2026/27	₹ 45000.00	₹ 5000.00	₹ 40000.00	24 Sep 2026	Draft
INV-2026-000001	2210013135065 kaif	B.Tech CSE - Semester 5 - 2026/27	₹ 45000.00	₹ 0.00	₹ 45000.00	22 Sep 2026	Draft

Concessions

Manage student fee concession requests.

+ New Concession

STUDENT	FEE HEAD	STRUCTURE	TYPE	VALUE	STATUS	CREATED	ACTION
2210013135065 kaif	Tuition Fee	B.Tech CSE - Semester 5 - 2026/27	Fixed Amount	₹ 5000.00	Approved	25 Sep 2026	View
2210013135065 kaif	Tuition Fee	B.Tech CSE - Semester 5 - 2026/27	Fixed Amount	₹ 10.00	Cancelled	24 Sep 2026	View